

pHPA: A Proactive Autoscaling Framework for Microservice Chain

Byungkwon Choi

KAIST

South Korea

Chunghan Lee

Toyota Motor Corporation

Japan

Jinwoo Park

KAIST

South Korea

Dongsu Han

KAIST

South Korea

ABSTRACT

Microservice is an architectural style that breaks down monolithic applications into smaller microservices and has been widely adopted by a variety of enterprises. Like the monolith, autoscaling has attracted the attention of operators in scaling microservices. However, most existing approaches of autoscaling do not consider microservice chain and severely degrade the performance of microservices when traffic surges. In this paper, we present pHPA, an autoscaling framework for the microservice chain. pHPA proactively allocates resources to the microservice chains and effectively handles traffic surges. Our evaluation using various open-source benchmarks shows that pHPA reduces 99%-tile latency and resource usage by up to 70% and 58% respectively compared to the most widely used autoscaler when traffic surges.

CCS CONCEPTS

• Software and its engineering → Cloud computing.

1 INTRODUCTION

Microservice is a software architectural style, which is widely adopted across various enterprises including Netflix [19], Amazon [21], and Airbnb [42]. According to the survey report from O'Reilly in 2020 [18, 20], 77% of 1,502 respondents who took on a technical role in the company said that they have introduced microservices to their businesses. The microservice architecture breaks traditional monolithic applications into multiple small components, which are called microservices. Microservices communicate with one another and form complex relationships. A request to the service goes through a chain of microservices to execute the request. It is reported that commercial applications have hundreds of microservices that exchange messages to each other [9, 32].

At the same time, resource autoscaling gains popularity for balancing the quality of service (QoS) and operating costs. Traffic dynamically changes due to various reasons. When traffic changes, allocating resources to cloud applications promptly is critical. If

resources are not provisioned enough in time, a service outage occurs [5, 11, 22, 38, 39]. For example, Amazon had faced a service outage due to the failure of provisioning additional servers on Prime Day in 2018 [11]. Microsoft and Zoom had experienced an outage due to the traffic surge stemming from the COVID-19 pandemic [22, 38]. To avoid such outages, some service operators overly provision resources, but this essentially leads to high operating costs and low utilization [6, 37, 54]. To balance operating costs and service quality, operators usually apply autoscaling in production [1, 2, 11, 23, 33]. Resource autoscaling detects changes in workload or resource usage and adjusts resource quota without human intervention. Autoscaling is promising, however, existing approaches suffer from limitations.

Most existing autoscalers [40, 53, 58] provision resources only after issues (e.g., long-tail latency, high resource utilization) happen. This approach becomes problematic when scaling microservice chains because resource provisioning takes time. Creating a microservice instance involves accessing a file system to fetch system libraries and data files and takes up to tens of seconds. Google Borg revealed that the median container startup latency is 25 seconds [56]. Recent work also reports that 90%-tile latency of microservice startup is about 15 seconds even with a state-of-the-art scheduler [44].

This provisioning time is propagated down through the microservice chain. When a microservice receives a request, for example, it subsequently transmits related requests to the following microservices in the same chain. If a microservice lacks resources, the incoming traffic to the microservice would be dropped and the microservice would not transmit requests to the following microservices. With the existing autoscalers, the further a microservice is located at the back of the chain, the slower it will perceive changes in the workload. Until the rearmost microservice in the chain does not experience a resources shortage, the service would continue to respond improperly. To avoid such disruption, some service operators manually provision resources even though they are using autoscaling schemes [11, 23]. Some operators also use schedule-based autoscaling to hide the resource provisioning time based on traffic history [17, 27–29], but this cannot deal with sudden traffic surges.

To overcome the limitation of the existing autoscalers, we present pHPA, a proactive autoscaling framework for microservice chain. pHPA first identifies the appropriate amount of resources for each microservice by using graph neural network (GNN) and gradient

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

APNet 2021, June 24–25, 2021, Shenzhen, China, China

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8587-9/21/06...\$15.00

<https://doi.org/10.1145/3469393.3469401>

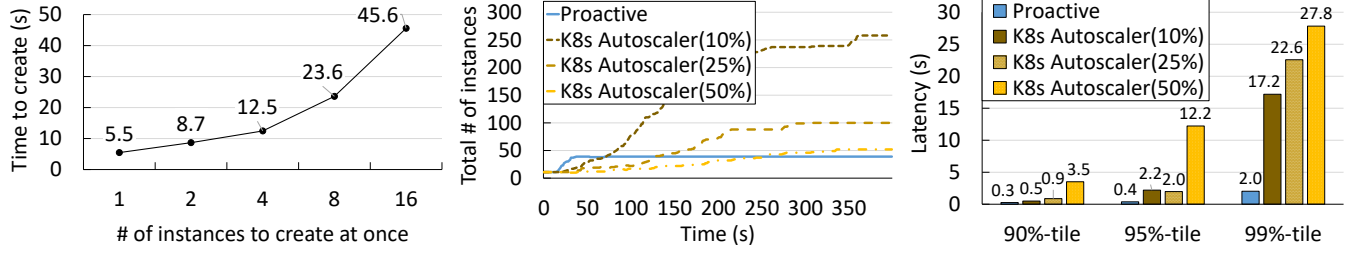


Figure 1: Time to create microservice instances. **Figure 2: Total Number of microservice instances when traffic surges.** **Figure 3: End-to-end latency when traffic surges.**

method. Then, pHPA proactively allocates resources to microservice chains and effectively handles traffic surges. We first demonstrate the ineffectiveness of an existing autoscaler during a traffic surge. We then present an approach that addresses the limitation. Our evaluation using various open-source benchmarks [4, 24, 31] and the prototype of pHPA demonstrates that pHPA reduces 99%-tile end-to-end latency and resource usage by up to 70% and 58% respectively compared to the most widely used autoscaler when traffic surges.

2 CASCADING EFFECT

A cascading effect is a phenomenon that subsequent microservices in a chain slowly perceive changes in the workload because of the instance creation delay of previous microservices in the chain. The cascading effect severely degrades the performance of microservices when traffic increases abruptly. As described in Section 6, most existing autoscalers do not consider the microservice chain and face the cascading effect. In this section, we describe the cascading effect based on Kubernetes (K8s) autoscaler [40] that is widely used [8, 10, 23, 26]. Then, we show how to avoid it.

K8s autoscaler operates with a pre-determined resource utilization threshold. When a microservice’s resource utilization reaches the threshold, the autoscaler creates more instances of the microservice to keep the utilization below a certain level¹. Service operators can control the threshold to adjust how quickly the autoscaler reacts to the changes in resource utilization. The autoscaler monitors the usage of the resources such as CPU or memory and independently creates instances for each microservice.

Cascading effect. Instance creation time is accumulated and propagated down through the microservice chains and we call this phenomenon the cascading effect. Figure 1 shows the time it takes to create instances². On average across microservices, it takes 5.5 seconds to create a single instance. When multiple instances are created at once, the creation time increases. Furthermore, in production settings, service operators set an interval (e.g., 15 seconds) of how often the autoscaler works to prevent the number of instances

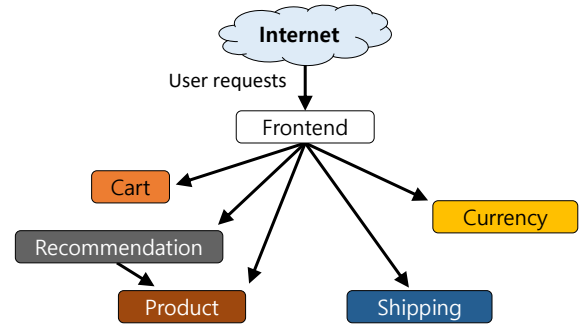


Figure 4: A microservice chain of an open source benchmark called Online Boutique [24]

from being fluctuated. This control interval makes the autoscaler much slower to perceive changes in resource utilization.

Due to the cascading effect, the further back a microservice is located within a chain, the longer it takes for the microservice to experience the changes in workload and resource usage. For example, Figure 4 shows one of the microservice chains in an open-source benchmark called Online Boutique [24]. This microservice chain is to get a cart page of the service. When ‘Frontend’ receives a request from an end-user, it first sends a request to the following microservice called ‘Currency’. ‘Frontend’ then sequentially sends a request to the successive microservice ‘Cart’ and so forth. Assume that ‘Currency’ receives an excessive amount of requests and the autoscaler creates more instances for ‘Currency’. During that time, ‘Cart’ is not aware of the change in the workload. After the additional instances of ‘Currency’ are created, ‘Cart’ perceives the increase of the workload. This happens sequentially to the following microservices.

Experimental result. We can observe this phenomenon in experiments. Figure 5 (upper graph) shows the workload that each microservice perceives when using K8s autoscaler. We transmit queries for the cart page at a rate of 300qps by using Vegeta [36]. While ‘Frontend’ perceives its peak traffic at 31s, ‘Cart’ starts handling its peak workload at 118s. It is because until enough number of instances for ‘Frontend’ is created the workload for ‘Cart’ does not reach the peak. The subsequent microservices see the peak workloads even further later at 155s. A naïve approach to reducing

¹One can vertically scale a microservice instance up by allocating more low-level resources such as CPU or memory. However, it is insufficient because the amount of resources allocated to an instance cannot get larger than the total amount of resources in the machine it is running on [55].

²We create instances of microservices in [24] on a single worker node and ignore the network delay to download the container images.

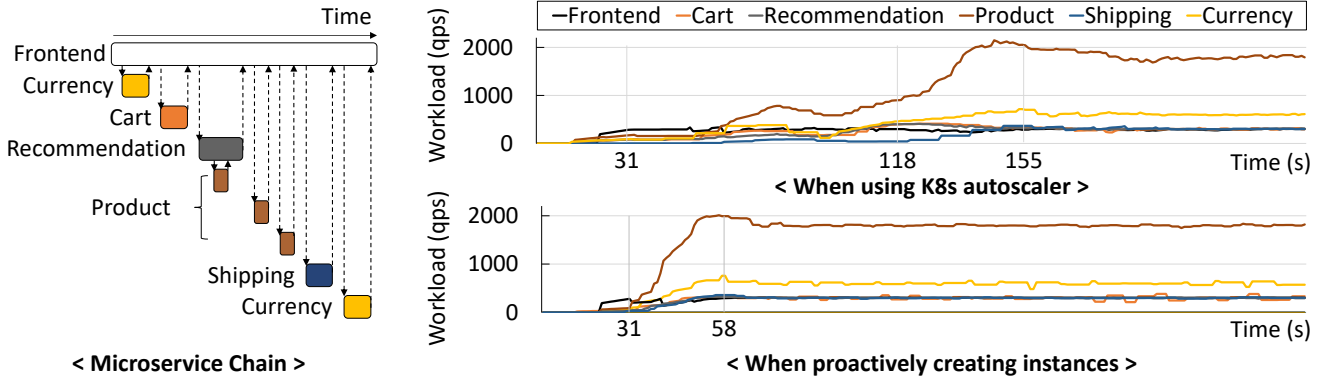


Figure 5: Microservice chain and workload that each microservice perceives when traffic surges.

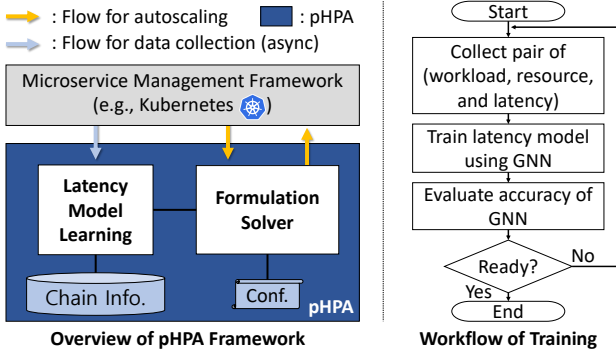


Figure 6: Overview and Workflow of pHPA framework

the delay caused by the cascading effect is to lower the resource utilization threshold, but it comes with a lot of costs. We run the same experiment while varying CPU utilization threshold from 10 to 50%. Figure 2 and 3 show the total number of microservice instances and the end-to-end latency, respectively. When we adjust the threshold from 50 to 10%, the 99%-tile latency decreases from 27.8 to 17.2 seconds but the total number of instances dramatically increases from 51 to 258.

Opportunity. When we create the instances for all microservices in a chain at once, we could avoid the cascading effect. We first transmit the cart queries and then manually create the heuristically-determined number of instances for each microservice. As shown in Figure 2 and 3, this approach ‘Proactive’ reduces the 99%-tile latency by 8.6 times compared to the 10% threshold setting of K8s autoscaler while creating 6.6 times less amount of the total instances. The time to reach the peak workload for all microservices is also similar to each other at 58s, as shown in Figure 5 (lower graph). If we can automatically determine the appropriate number of instances, we can develop an autoscaler that avoids the cascading effect.

3 DESIGN AND IMPLEMENTATION

Approach. Our goal is to automatically identify the appropriate number of instances that satisfies latency SLO and this can be

formulated as:

$$\min_{\vec{r}} \sum_{r \in \vec{r}} r \quad (1)$$

$$s.t. L(\vec{r}, w) \leq \text{Latency SLO} \quad (2)$$

where $\vec{r}$ is the number of instances for each microservice; w is workload (e.g., queries per sec); and $L(\vec{r}, w)$ is the end-to-end tail latency of microservices. We must solve the formula in real-time because workload changes continuously. However, it is non-trivial because trying possible combinations of resources in real-time is infeasible. If we change the number of instances, it affects the performance of microservices. In addition, the search space is also very large because there are tens [24, 31, 45] or hundreds [9, 32] of microservices.

To overcome this challenge, we model tail latency by using a graph neural network (GNN). $L(\vec{r}, w)$ is a complex black-box function and structured in the form of a graph. Estimating $L(\vec{r}, w)$ by assuming latency is drawn from a certain distribution is infeasible because of the multimodality of distribution from which latency of microservices is drawn and the complexity of the microservice chain. GNN is known to be scalable when modeling graph-structured workloads [43, 51, 52] like microservice chain. By leveraging GNN, we model $L(\vec{r}, w)$. With the trained latency model, we solve the formula by using a gradient method. To apply gradient method, we transform the formulation.

pHPA design. Figure 6 (left side) illustrates an overview of the pHPA framework. In the beginning, pHPA exploits an existing approach such as K8s autoscaler until training is done. Then, pHPA periodically solves the formulation by using the trained model and gradient method.

It consists of two modules. First, *Latency Model Learning* module collects samples from a microservice management framework and trains a latency model using GNN. A sample consists of traffic that end-users transmit to an application, the number of microservice instances, and tail latency (e.g., 90%-tile). The architecture of the GNN model is constructed based on the microservice chain structure. The GNN model takes workload and the number of instances of microservices as input and infers tail latency. *Latency Model Learning* module continuously scrapes samples and trains the GNN model until the accuracy of GNN exceeds a certain threshold, as

App	Underestimate Points		Overestimate Perc.	
	Linear	PPGPR	Linear	PPGPR
Robot Shop	50%	0%	18%	22%
Bookinfo	50%	15%	64%	16%
Online Boutique	58%	5%	150%	23%

Table 1: CPU usage estimation ratio when using PPGPR and linear regression. ‘Overestimate Perc.’ indicates average percentage error.

shown in Figure 6 (right side). This module divides the collected samples by training and test sets and evaluates the accuracy of the model. When training is done, pHPA stops exploiting the existing autoscaler and runs the next module.

Formulation Solver identifies the appropriate number of instances for each microservice by using the trained GNN model and gradient method. It first scrapes the frontend workload from the management framework. We combine Equation 1 and 2 to apply gradient method as follows:

$$\min_{\vec{r}} \sum_{r \in \vec{r}} r - \lambda \times \max(L(\vec{r}, w) - SLO, 0) \quad (3)$$

where λ is a penalty term that is applied when latency SLO is violated. We use a smooth max function [30] here and $L(\vec{r}, w)$ is modeled by using GNN so that Equation 3 is end-to-end differentiable and gradient method can be applied. *Formulation Solver* module solves the transformed formulation by using gradient method and identifies the appropriate number of instances for each microservice. Finally, the solution is applied to microservice applications.

pHPA prototype. We implement a prototype that calculates the number of instances based on CPU usage. It is because the work is underway to implement and evaluate the design described in Section 3. The prototype predicts CPU usage for each microservice by using Parametric Predictive Gaussian Process Regression (PPGPR) [48] and calculates the number of microservice instances by using $\text{ceil}(\text{CPU usage} / (\text{quota} \times (1 - \text{margin})))$. The prototype estimates CPU usage of each microservice based on the frontend workload and creates the instances proactively to avoid the cascading effect. PPGPR is known to be suitable at modeling stochastic data such as CPU usage [48]. quota indicates resource quota of a single microservice instance. margin is a ratio of how much CPU quota margin to give. We leverage Prometheus [25] and Linkerd [15] to collect CPU usage and workload and Jaeger [12] for the information of microservice chains. We implement PPGPR using GPytorch [35]. The prototype is implemented in 3.2K lines of Python code. We call this prototype pHPA in the next section.

4 PRELIMINARY EVALUATION

We evaluate pHPA using three open-source applications including *Bookinfo* [4], *Robot Shop* [31], and *Online Boutique* [24]. Microservice chain of the applications is illustrated in Figure 4 and 7, respectively. All microservices are deployed using Docker containers. We use Kubernetes [13] as the underlying container orchestration framework. We run Kubernetes clusters on 7 machines with two Intel E5-2650 CPUs and 128GB of memory. We use one machine for

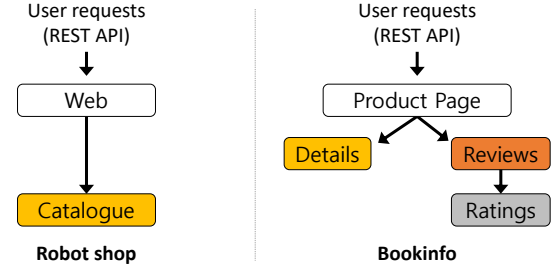


Figure 7: Microservice chain of Robot Shop [31] and Bookinfo [4]

Kubernetes master node and 6 for worker nodes. We run the load generators on a separate machine. Unless specifically noted, we allocate 0.3 CPU quota per microservice instance and set margin of pHPA and the CPU utilization threshold of K8s autoscaler to 0.5 and 25%, respectively.

4.1 Resource Usage Prediction Analysis

We evaluate how accurately pHPA predicts CPU usages. We run the applications and collect pairs of workload and CPU usage for each microservice while varying workload from 50 to 300 qps using Vegeta [36]. We divide it by train(80%) and test(20%) set and train the resource usage model for each microservice using PPGPR. We use the combination of linear and RBF for PPGPR kernel function. We compare the results of PPGPR with that of linear regression. We use the upper bound of 95% confidence interval as the estimation value. PPGPR takes 6 μ s on average to predict CPU usage.

Table 1 shows the result. PPGPR underestimates CPU usage only for up to 15% of data points across the applications while linear regression underestimates for 50 to 58% of data points. Underestimating resource usage is critical because the lack of resources would lead to severe degradation in performance. When we increase the intercept value of the y-axis for linear regression to make the underestimation ratio to be the same as that of PPGPR, linear regression overestimates CPU usage by 18 to 150% on average across the applications while PPGPR overestimates by 15 to 23% on average.

4.2 End-to-end Evaluation

We compare pHPA with K8s autoscaler. Each microservice has a single instance at the beginning. The control interval of both pHPA and K8s autoscaler is set to 10 seconds. By using Vegeta [36], we send 1000qps, 200qps, and 300qps of workload to *Robot shop*, *Bookinfo*, and *Online Boutique*, respectively. We pre-train PPGPR of pHPA. We count the total number of microservice instances every second and measure the end-to-end latency of requests.

Figure 8 and 9 show the total number of instances and end-to-end latency, respectively. pHPA reduces 99%-tile latency by 36 to 70% while creating 1.1 to 2.4 times less number of instances at the end across the applications, compared to K8s autoscaler. The longer a microservice chain is, the more pHPA improves latency. pHPA reduces 99%-tile latency of *Online Boutique* by 70% while reducing that of *Robot Shop* by 50%. As shown in Figure 8 (b), the total number of instances created by K8s autoscaler increases to 165 in the middle

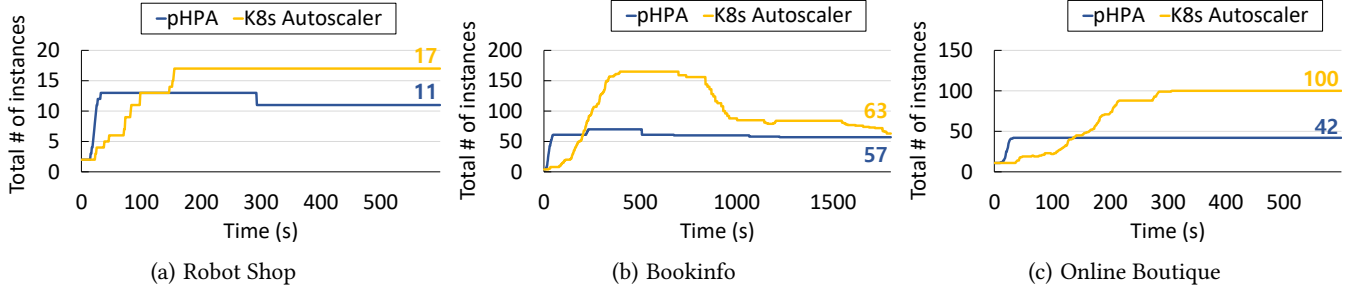


Figure 8: Total # instances of microservice applications when using pHPA and Kubernetes autoscaler.

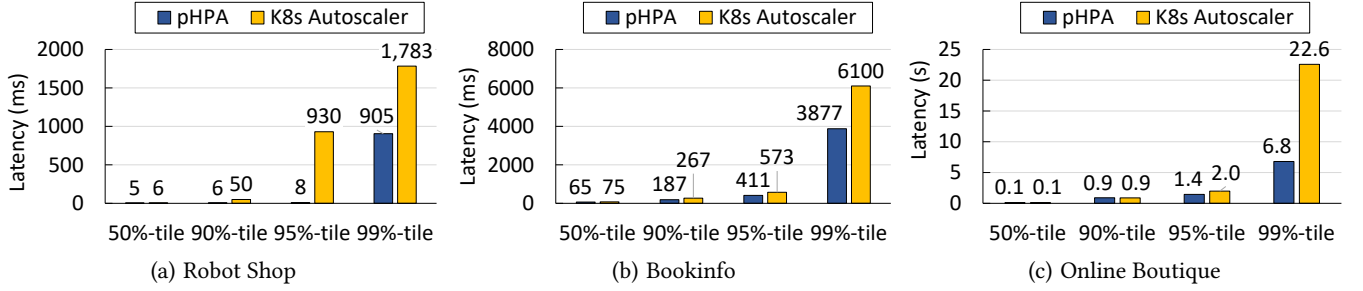


Figure 9: End-to-end latency of microservice applications when using pHPA and Kubernetes autoscaler.

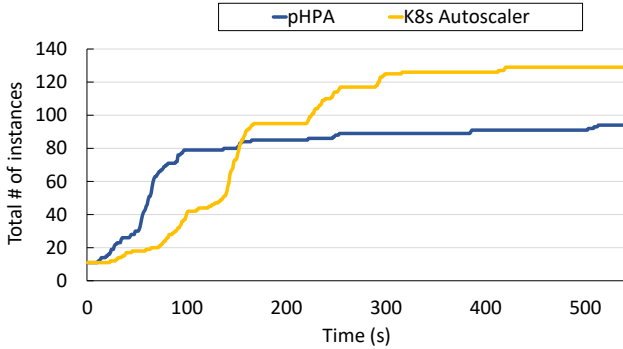


Figure 10: Resource usage when using dynamic workload

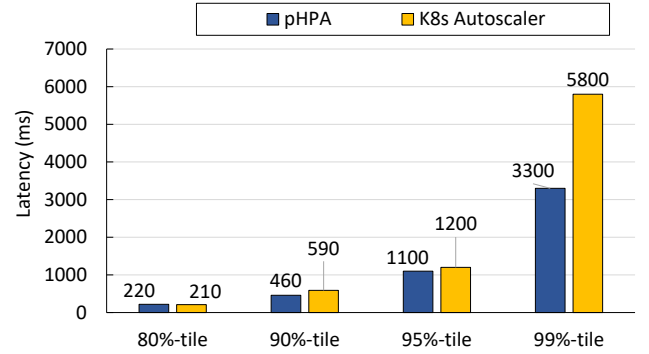


Figure 11: Latency when using dynamic workload

and then decreases. It is because one of the microservices in *Bookinfo* is implemented in Java and consumes a lot of CPU cycles in the beginning to convert Java bytecodes to machine code [7]. pHPA also experiences the same, but the conversion occurs in parallel and pHPA delivers better latency even with a smaller number of instances.

Evaluation with dynamic workload. We evaluate pHPA in a more realistic environment. We use Locust [16] to generate workload. Locust spawns multiple user threads, each of which sends various types of requests in a predefined order. The Locust thread randomly waits for up to 3 seconds until it sends the next request

to mimic the actual user behavior. We create 10 threads every second until the total number of threads reaches 500. As shown in Figure 10 and 11, pHPA reduces 99%-tile latency by 43% while creating 1.4 times less number of instances at the end compared to K8s autoscaler.

Cost vs QoS. We compare pHPA with K8s autoscaler in terms of how well it can balance the tradeoff between cost and service quality. We vary the CPU utilization threshold of K8s autoscaler from 25 to 50% and margin of pHPA from 0.5 to 0.75 while sending workload by using Locust. We calculate the operating cost by following the pricing plan of AWS Fargate [3]. As shown in Figure 12, pHPA

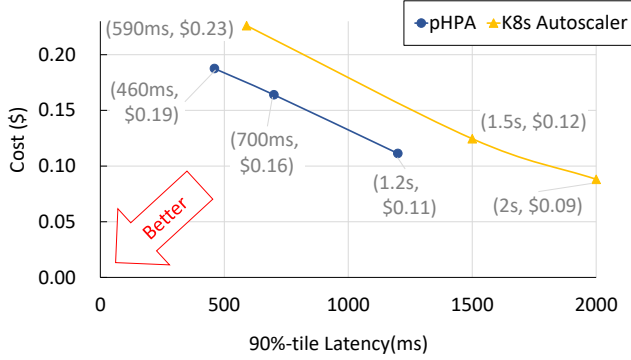


Figure 12: Tradeoff between latency and cost of K8s autoscaler and pHPA.

controls the balance between cost and latency better than K8s autoscaler.

5 DISCUSSION AND FUTURE WORK

Considering multiple microservice chains. pHPA optimizes resource usage for a single microservice chain. However, there exist multiple chains in an application. For example, an open-source benchmark called Online Boutique exposes six APIs and each API composes a different microservice chain. And also each chain shares some microservices.

To optimize resource usages for multiple chains, one can naïvely apply the pHPA approach by adding more max terms to Equation 3. As more terms are added, however, the objective function of Equation 3 gets more complex and gradient method becomes more likely to be stuck in the local optimum. The problem is that many optimization algorithms such as gradient method are devised under the assumption of convexity [14], however, it cannot be guaranteed that the objective function is convex. Recently, some automation approaches [41, 50] have been introduced to overcome the limitation. The core idea of those works is that they learn how to iteratively update point in the domain by leveraging reinforcement learning. We leave the adoption of it to optimize for multiple microservice chains as future work.

6 RELATED WORKS

Autoscaling. There have been a number of frameworks that autoscale cloud applications [46, 53, 55, 58, 59]. To the best of our knowledge, none of the existing autoscalers could avoid the cascading effect. FIRM [53] and MIRAS [58] are an RL-based autoscaling framework. Both works handle only microservices that experience resource shortages and this faces the cascading effect. FIRM [53] uses SVM to identify microservices critical to latency SLO violations and adjusts the quota of various resources through a policy learned by RL algorithm. Because FIRM allocates more resources to microservices that cause latency SLO violations, FIRM does not handle subsequent microservices in a chain until previous microservices in the chain have enough resources. MIRAS [58] learns a policy that allocates more resources to microservices with a long request queue. MIRAS does not handle subsequent microservices in a chain

until the queue is long enough. ATOM [46] uses queueing model and genetic algorithm. ATOM adjusts resources for microservices at once, however, ATOM still suffers from traffic surges because of the long control interval caused by expensive genetic algorithm.

Container startup latency. Container startup latency is the main cause of developing the cascading effect into a severe problem. S. Fu et al [44] present a scheduling scheme that leverages dependencies between layers of the container images to reduce container startup time. Their scheduler has been adopted into the mainline Kubernetes codebase, however, it still delivers slow startup time (90%-tile startup latency is about 15 seconds). Slacker [47] lazily pulls the contents of the container image and improves startup latency, however, it requires modifications to the linux kernel and the use of a proprietary NFS server, which are non-trivial. Some approaches [34, 49, 57] reuse containers to reduce startup delay. However, such strategies would bring about the excessive use of resources such as memory (90%-tile size of container images in Docker Hub is 1.3GB [60]) and the expensive charge of cloud services [44].

7 CONCLUSION

The role of autoscaling is to save resource usage while satisfying service quality requirement without human intervention. Existing autoscalers suffer from the cascading effect and deliver poor performance in scaling microservice chains when traffic surges. In this paper, we argue that an autoscaler for microservice chain must be designed to operate proactively. We present pHPA, an autoscaling framework that proactively allocates resources for microservice chains. Our preliminary evaluation using the prototype of pHPA demonstrates that pHPA reduces 99%-tile latency by 43% and resource usage by 27% at the same time when traffic changes, compared to the autoscaler that is most widely used in production.

REFERENCES

- [1] [n. d.]. Accelerate and Autoscale Deep Learning Inference on GPUs with KFServing. https://www.youtube.com/watch?v=LX_AWimNOhU. ([n. d.]).
- [2] [n. d.]. Auto Scaling Production Services on Titus. <https://netflixtechblog.com/auto-scaling-production-services-on-titus-1f3cd49f5cd7/>. ([n. d.]).
- [3] [n. d.]. AWS Fargate. <https://aws.amazon.com/fargate>. ([n. d.]).
- [4] [n. d.]. Bookinfo Application by Istio. <https://istio.io/latest/docs/examples/bookinfo/>. ([n. d.]).
- [5] [n. d.]. Canada's immigration website crashed due to traffic surge. <https://www.ctvnews.ca/canada/canada-s-immigration-website-crashed-due-to-traffic-surge-1.3152744>. ([n. d.]).
- [6] [n. d.]. Cloud Waste To Hit Over 14 Billion in 2019. <https://devops.com/cloud-waste-to-hit-over-14-billion-in-2019/>. ([n. d.]).
- [7] [n. d.]. Compiling and running a Java program. <https://books.google.co.kr/books?id=8kD8j3FCk58C&pg=PR20>. ([n. d.]).
- [8] [n. d.]. Configuring horizontal Pod autoscaling in GKE. <https://cloud.google.com/kubernetes-engine/docs/how-to/horizontal-pod-autoscaling>. ([n. d.]).
- [9] [n. d.]. Examples and types of microservices. <https://www.itrelease.com/2018/10/examples-and-types-of-microservices/>. ([n. d.]).
- [10] [n. d.]. Horizontal Pod Autoscaler in AWS. <https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html>. ([n. d.]).
- [11] [n. d.]. Internal documents show how Amazon scrambled to fix Prime Day glitches. <https://www.cnbc.com/2018/07/19/amazon-internal-documents-what-caused-prime-day-crash-company-scramble.html>. ([n. d.]).
- [12] [n. d.]. Jaeger: open source, end-to-end distributed tracing. <https://www.jaegertracing.io/>. ([n. d.]).
- [13] [n. d.]. Kubernetes: Production-Grade Container Orchestration. <https://kubernetes.io/>. ([n. d.]).
- [14] [n. d.]. Learning to Optimize with Reinforcement Learning. <https://bair.berkeley.edu/blog/2017/09/12/learning-to-optimize-with-rl/>. ([n. d.]).

- [15] [n. d.]. Linkerd: The world's lightest, fastest service mesh. <https://linkerd.io/>. ([n. d.]).
- [16] [n. d.]. Locust: An open source load testing tool. <https://locust.io/>. ([n. d.]).
- [17] [n. d.]. Managing traffic spikes with schedule-based auto scaling. <https://cloud.ibm.com/docs/virtual-servers?topic=virtual-servers-managing-schedule-based-auto-scaling>. ([n. d.]).
- [18] [n. d.]. Microservice architecture growing in popularity, adopters enjoying success. <https://www.itproportal.com/news/microservice-architecture-growing-in-popularity-adopters-enjoying-success/>. ([n. d.]).
- [19] [n. d.]. Microservices - Netflix Techblog. <https://netflixtechblog.com/tagged/microservices>. ([n. d.]).
- [20] [n. d.]. Microservices Adoption in 2020. <https://www.oreilly.com/radar/microservices-adoption-in-2020/>. ([n. d.]).
- [21] [n. d.]. Microservices at Amazon. <https://www.slideshare.net/apigee/i-love-apis-2015-microservices-at-amazon-54487258>. ([n. d.]).
- [22] [n. d.]. Microsoft Confirms March Azure Outage Due to COVID-19 Strains. <https://visualstudiomagazine.com/articles/2020/04/13/azure-outage.aspx>. ([n. d.]).
- [23] [n. d.]. Multi-Tenancy Kubernetes on Bare Metal Servers. [https://devview.kr/data/devview/2019/presentation/\[231\]+Multi-Tenancy+Kubernetes+on+Bare+Metal+Servers.pdf](https://devview.kr/data/devview/2019/presentation/[231]+Multi-Tenancy+Kubernetes+on+Bare+Metal+Servers.pdf) (16p). ([n. d.]).
- [24] [n. d.]. Online Boutique by Google. <https://github.com/GoogleCloudPlatform/microservices-demo>. ([n. d.]).
- [25] [n. d.]. Prometheus - Monitoring system & time series database. <https://prometheus.io/>. ([n. d.]).
- [26] [n. d.]. Scale applications in Azure Kubernetes Service (AKS). <https://docs.microsoft.com/en-us/azure/aks/tutorial-kubernetes-scale>. ([n. d.]).
- [27] [n. d.]. Scaling based on schedules. <https://cloud.google.com/compute/docs/autoscaler/scaling-schedules>. ([n. d.]).
- [28] [n. d.]. Schedule-Based Autoscaling. <https://docs.oracle.com/en-us/iaas/Content/Compute/Tasks/autoscalinginstancepools.htm#time>. ([n. d.]).
- [29] [n. d.]. Scheduled scaling for Amazon EC2 Auto Scaling. https://docs.aws.amazon.com/autoscaling/ec2/userguide/schedule_time.html. ([n. d.]).
- [30] [n. d.]. Smooth maximum. https://en.wikipedia.org/wiki/Smooth_maximum. ([n. d.]).
- [31] [n. d.]. Stan's Robot Shop by Instana. <https://www.instana.com/blog/stans-robot-shop-sample-microservice-application/>. ([n. d.]).
- [32] [n. d.]. The Story of Netflix and Microservices. <https://www.geeksforgeeks.org/the-story-of-netflix-and-microservices/>. ([n. d.]).
- [33] [n. d.]. Throughput Autoscaling: Dynamic sizing for facebook.com. <https://www.facebook.com/watch/?v=378764619812456>. ([n. d.]).
- [34] [n. d.]. Understanding Container Reuse in AWS Lambda. <https://aws.amazon.com/blogs/compute/container-reuse-in-lambda/>. ([n. d.]).
- [35] [n. d.]. Variational and Approximate GPs. https://docs.gpytorch.ai/en/v1.1.1/examples/04_Variational_and_Approximate_GPs/. ([n. d.]).
- [36] [n. d.]. Vegeta: a versatile HTTP load testing tool. <https://github.com/tseart/vegeta>. ([n. d.]).
- [37] [n. d.]. Wasted Cloud Spend to Exceed 17.6 Billion in 2020, Fueled by Cloud Computing Growth. <https://jaychapel.medium.com/wasted-cloud-spend-to-exceed-17-6-billion-in-2020-fueled-by-cloud-computing-growth-7c8f81d5c616>. ([n. d.]).
- [38] [n. d.]. Zoom suffers "partial outage" amid home working surge. <https://www.datacenterdynamics.com/en/news/zoom-suffers-partial-outage-amid-home-working-surge/>. ([n. d.]).
- [39] [n. d.]. "Google Down": How Users Experienced Google's Major Outage. <https://www.semrush.com/blog/google-down-how-users-experienced-google-major-outage/>. ([n. d.]).
- [40] 2021. Horizontal Pod Autoscaler of Kubernetes. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. (2021).
- [41] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando De Freitas. 2016. Learning to learn by gradient descent by gradient descent. *arXiv preprint arXiv:1606.04474* (2016).
- [42] Melanie Cebula. 2017. Airbnb, From Monolith to Microservices: How to Scale Your Architecture. <https://www.youtube.com/watch?v=N1BWMW9NEQc>. (2017).
- [43] Hanjun Dai, Elias B Khalil, Yuyu Zhang, Bistra Dilkina, and Le Song. 2017. Learning combinatorial optimization algorithms over graphs. *arXiv preprint arXiv:1704.01665* (2017).
- [44] Silvery Fu, Radhika Mittal, Lei Zhang, and Sylvia Ratnasamy. 2020. Fast and efficient container startup at the edge via dependency scheduling. In *3rd {USENIX} Workshop on Hot Topics in Edge Computing (HotEdge 20)*.
- [45] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 3–18.
- [46] Alim Ul Gias, Giuliano Casale, and Murray Woodside. 2019. ATOM: Model-driven autoscaling for microservices. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1994–2004.
- [47] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. 2016. Slacker: Fast distribution with lazy docker containers. In *14th {USENIX} Conference on File and Storage Technologies ({FAST} 16)*. 181–195.
- [48] Martin Jankowiak, Geoff Pleiss, and Jacob R. Gardner. 2020. Parametric Gaussian Process Regressors. In *Proceedings of the 37th International Conference on Machine Learning, Online, PMLR*.
- [49] Eric Jonas, Qifan Pu, Shivaram Venkataraman, Ion Stoica, and Benjamin Recht. 2017. Occupy the cloud: Distributed computing for the 99%. In *Proceedings of the 2017 Symposium on Cloud Computing*. 445–451.
- [50] Ke Li and Jitendra Malik. 2016. Learning to optimize. *arXiv preprint arXiv:1606.01885* (2016).
- [51] Zhuwen Li, Qifeng Chen, and Vladlen Koltun. 2018. Combinatorial optimization with graph convolutional networks and guided tree search. *arXiv preprint arXiv:1810.10659* (2018).
- [52] Hongzi Mao, Malte Schwarzkopf, Shailesh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. 2019. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication*. 270–288.
- [53] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishanker K. Iyer. 2020. FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 805–825. <https://www.usenix.org/conference/osdi20/presentation/qiu>
- [54] Charles Reiss, Alexey Tumanov, Gregory R Ganger, Randy H Katz, and Michael A Kozuch. 2012. Heterogeneity and dynamics of clouds at scale: Google trace analysis. In *Proceedings of the third ACM symposium on cloud computing*. 1–13.
- [55] Krzysztof Rzadca, Paweł Findeisen, Jacek Świderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmerek, Paweł Krzysztof Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. 2020. Autopilot: Workload Autoscaling at Google Scale. In *Proceedings of the Fifteenth European Conference on Computer Systems*. <https://dl.acm.org/doi/10.1145/3342195.3387524>
- [56] Abhishek Verma, Luis Pedrosa, Madhukar R. Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In *Proceedings of the European Conference on Computer Systems (EuroSys)*. Bordeaux, France.
- [57] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. 2018. Peeking behind the curtains of serverless platforms. In *2018 {USENIX} Annual Technical Conference ({USENIX} {ATC} 18)*. 133–146.
- [58] Z. Yang, P. Nguyen, H. Jin, and K. Nahrstedt. 2019. MIRAS: Model-based Reinforcement Learning for Microservice Resource Allocation over Scientific Workflows. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. 122–132. <https://doi.org/10.1109/ICDCS.2019.00021>
- [59] Guangba Yu, Pengfei Chen, and Zibin Zheng. 2019. Microscaler: Automatic scaling for microservices with an online learning approach. In *2019 IEEE International Conference on Web Services (ICWS)*. IEEE, 68–75.
- [60] Nannan Zhao, Vasily Tarasov, Hadeel Albahar, Ali Anwar, Lukas Rupprecht, Dimitrios Skourtis, Amit S Warke, Mohamed Mohamed, and Ali R Butt. 2019. Large-scale analysis of the docker hub dataset. In *2019 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 1–10.